

Audio-Foto-Pipeline

Projektdokumentation

Python for Beginners FS26 | Fabio Roos | HSLU | 28. Mai 2026

1 Problembeschreibung

Bei Baubegehungen — Infrastrukturprojekte, Schadenserfassung, Baustellenprotokoll — erfasst ein Techniker Befunde vor Ort. Die manuelle Nachbearbeitung (Notizen abtippen, Fotos den richtigen Befunden zuordnen, Dokument erstellen) ist zeitaufwändig und fehleranfällig. In der Praxis führt dieser Aufwand dazu, dass Dokumentationen unvollständig bleiben oder gänzlich ausbleiben — Befunde geraten in Vergessenheit.

Der Ansatz entstand aus direkten Gesprächen mit Projektleitern, die täglich auf Baustellen arbeiten. Ihre geschilderten Probleme bildeten die Grundlage für die entwickelte Lösung — mit dem Ziel, dass sie tatsächlich im Unternehmen eingesetzt werden kann.

Der Autor hat eine Ausbildung als Zeichner Fachrichtung Ingenieurbau absolviert und arbeitet heute im Ingenieurbüro Emch+Berger WSB AG — zuerst im Ingenieurbau, mittlerweile in der IT-Abteilung. Diese Kombination ermöglichte es, technische Anforderungen aus dem Baubereich direkt in eine praxisnahe Softwarelösung zu übersetzen.

Die Pipeline ist generell einsetzbar und läuft vollständig lokal; nur der Transkriptions-API-Aufruf benötigt Internet. Der Techniker spricht den Befund kurz per Audio an und macht ein Foto — den Rest übernimmt die Pipeline automatisch.

2 Lösungsansatz

Die Pipeline verarbeitet Audio und Fotos in vier aufeinanderfolgenden Modulen:

- **match.py:** Liest EXIF-Zeitstempel der Fotos und den Aufnahmezeitpunkt der Audiodateien. Fotos werden den Audio-Abschnitten chronologisch zugeordnet.
- **transcribe.py:** Sendet Audiodateien an die OpenAI Whisper API (bis 25 MB). Für kurze, prägnante Aufnahmen einer Begehung vollständig ausreichend.
- **format.py:** Bereinigt das Transkript regelbasiert: Füllwörter entfernen, Segmente nach Sprachpausen gruppieren. Kein LLM, nur Textanalyse.
- **export.py:** Erstellt ein Word-Dokument (.docx) mit Titelseite, Notizseiten und eingebetteten Fotogalerien. Struktur und Stil wurden AI-gestützt erarbeitet.

Die Anwendungsmöglichkeiten sind vielfältig: Baustellenfortschritt dokumentieren, Schäden erfassen, Zustandsaufnahmen durchführen oder Begehungsprotokolle erstellen — überall dort, wo vor Ort Befunde mit Bild und Ton festgehalten werden sollen.

Die Eingabe erfolgt über Ordner: Mehrere Aufnahmen und Fotos eines Rundgangs werden in einem Durchgang verarbeitet. Fotos werden anhand ihrer Zeitstempel automatisch korrekt in die Timeline eingeordnet.

Beispieldaten: Im Projektordner unter `data/audio/` und `data/fotos/` liegen reale Aufnahmen einer Zustandsaufnahme einer Quartierstrasse im Quartier Konstanz in Rothenburg. Der Dozent kann die Pipeline sofort starten — beim Abfragen der Ordnerpfade genügt ein Drücken der Eingabetaste, um die enthaltenen Beispieldaten zu verwenden. Das generierte Protokoll liegt dem Abgabeordner ebenfalls bei.

3 Herausforderungen und Erfahrungen

Zeitstempel-Zuverlässigkeit

Zunächst wurde versucht, den Aufnahmezeitpunkt aus dem MP4-Metadaten-Tag zu lesen. Dies erwies sich als unzuverlässig: Beim Übertragen von Dateien werden diese Metadaten oft überschrieben. Lösung: ein Audio-Recorder, der den Aufnahmezeitpunkt direkt im Dateinamen speichert (YYYYMMDD-HHMMSS.m4a). Bei falschem Format fällt der Code auf die Dateisystem-mtime zurück — mit Warnung.

API-Key-Handling

Aus Sicherheitsgründen wird der API-Key normalerweise in einer .env-Datei gespeichert. Für die Abgabe wurde er in config.json eingetragen, damit der Dozent das Projekt direkt starten kann. Der Code prüft zuerst config.json und nimmt .env als Fallback — beide Workflows bleiben erhalten.

Arbeit mit AI und eigene Verantwortung

Der gesamte Code wurde gemeinsam mit Claude AI erarbeitet und reviewt. Dabei zeigte sich, dass offensichtliche Fehler trotz AI-Review nicht immer erkannt wurden — eine wichtige Erfahrung: Man kann nicht blind auf AI vertrauen. Es braucht ein eigenes Verständnis des Codes, kritisches Hinterfragen und aktives Mitdenken. Claude hat ausserdem massgeblich bei der Code-Strukturierung (PEP 8) und der Word-Ausgabe geholfen.

4 Handhabung — Schritte für den Dozenten

```
python -m venv .venv
source .venv/bin/activate # macOS/Linux | .venv\Scripts\activate (Windows)
pip install -r requirements.txt
# openai_api_key in config.json eintragen (Feld bereits vorbereitet)
python main.py run
```

Die Pipeline startet interaktiv. Beim Abfragen der Ordnerpfade genügt ein Drücken der **Eingabetaste** — die Beispieldaten in data/audio/ und data/fotos/ werden automatisch verwendet. Tests: `python -m pytest tests/`

Hilfsmittel und Quellen

Claude AI (Anthropic) — Unterstützung bei Codeentwicklung, Code-Review, Strukturierung und Formatierung des Word-Outputs. OpenAI Whisper API — Speech-to-Text-Transkription der Audiodateien. Alle eingesetzten Python-Bibliotheken sind in requirements.txt aufgeführt.